

블록체인 인덱서 구현과 운영에 대한 심층 분석 보고서

Executive Summary

블록체인 인덱서는 원장에 순차적으로 기록되는 블록·트랜잭션·이벤트·상태 변화를 애플리케이션이 읽기 쉬운 형태로 추출하고, 스키마화하며, 데이터베이스나 검색 인덱스에 적재한 뒤, GraphQL·SQL·REST·gRPC 같은 질의 인터페이스로 제공하는 데이터 인프라 계층이다. Polkadot 문서는 이를 “블록체인 데이터용 검색 엔진”에 비유하며, The Graph·SubQuery·Blockscout·NEAR·Solana 생태계의 공식 문서도 모두 같은 방향의 ETL 및 질의 가속 레이어로 설명한다. 1

인덱서가 필요한 이유는 네 가지로 압축된다. 첫째, **성능**이다. 블록체인 데이터는 본질적으로 순차적이고 분산되어 있어 다중 블록·과거 상태·집계 질의를 직접 RPC로 수행하면 느리고 비싸다. 둘째, **사용자 경험**이다. 지갑, 탐색기, 분석 대시보드, 게임, DeFi 앱은 “지금 막 발생한 이벤트”와 “역사적 집계 결과”를 함께 요구한다. 셋째, **데이터 접근성**이다. 계약/프로그램별 도메인 스키마, 검색, 페이징, 집계, 전문검색(full-text search)을 제공하려면 원시 RPC만으로는 부족하다. 넷째, **분산성**이다. 인덱서가 잘 설계되면 특정 호스트의 RPC 사업자 의존을 줄이고, 반대로 잘못 설계되면 체인의 탈중앙성을 애플리케이션 계층에서 다시 중앙화할 수 있다. The Graph는 이를 네트워크형 인덱서 시장과 슬래싱·서명된 영수증으로 보완하고, Ethereum 문서는 자체 풀노드가 여전히 가장 신뢰 최소화(trustless)된 접근 방식이라고 설명한다. 2

실무적으로는 네 가지 구현 패턴이 반복된다. **풀노드 기반 pull 모델**은 Geth/Reth/nearcore처럼 노드에서 직접 읽으며 정합성이 좋고 분산적이지만, 노드 운영과 스토리지 부담이 크다. **이벤트 스트리밍 모델**은 Solana Geyser, Ethereum WebSocket subscribe, Kafka 기반 파이프라인처럼 지연시간을 줄이는 대신 재전송·중복·체인 재구성(reorg) 처리가 중요하다. **트랜잭션/이벤트 스캔 모델**은 EVM logs, SubQuery dictionary, Blockscout 실시간·캐치업 인덱서처럼 선택적 필터링으로 효율을 높인다. **상태 스냅샷/파일 레이크 모델**은 NEAR Lake나 Firehose처럼 원시 체인 데이터를 파일·오브젝트 스토리지·gRPC 스트림으로 외부화해 대규모 재처리와 병렬화를 쉽게 만든다. 3

아키텍처 관점에서 가장 안정적인 설계는 **원시 데이터 계층과 질의용 물질화 뷰(materialized projection) 계층을 분리**하는 것이다. 소스 수집기(source collector)는 풀노드, WebSocket, Geyser, Firehose, S3/오브젝트 스토리지에서 데이터를 가져오고, 파서/디코더는 ABI·IDL·프로토콜 스키마를 이용해 이벤트를 구조화하며, 저장 계층은 보통 PostgreSQL을 기준 저장소(system of record)로 두고, RocksDB/Redis를 캐시·상태 저장소로, Elasticsearch/OpenSearch를 검색·집계용 보조 인덱스로 쓴다. Kafka 같은 로그 버스는 재처리와 역압(backpressure) 제어, 수평 확장을 쉽게 해준다. 4

사례 분석에서는 다섯 가지 대표 축이 명확하다. **The Graph**는 Graph Node + Postgres + 샤딩 + 전용 쿼리 노드 + Firehose/Substreams 병렬화를 통해 네트워크형 인덱싱을 구현한다. **NEAR**는 nearcore 기반 Indexer와 과거의 NEAR Lake, 그리고 ReadRPC 계열 구조로 상태변화·트랜잭션 인덱서를 분리한다. **Polkadot/Substrate + SubQuery**는 handler filter, dictionary, worker thread, block confirmation 회복을 통해 범용 멀티체인 인덱서를 제공한다. **Solana**는 Geyser 플러그인을 Validator 옆에 붙여 PostgreSQL/Kafka/gRPC로 데이터 외부화를 수행하고, Yellowstone gRPC가 저지연 스트리밍 계층을 담당한다. **Ethereum + Blockscout/Geth**는 archive/full node, eth_subscribe, tracing, realtime/catchup dual indexer, 체인별 secondary fetcher 구조로 탐색기와 API를 만든다. 5

성능 수치는 프로젝트마다 공개 수준이 다르지만, 실무에 도움이 되는 몇 가지 값은 분명하다. The Graph는 Substreams 기반으로 일부 서브그래프가 **100배 이상 빠르게 동기화**될 수 있다고 설명한다. Firehose는 신규 블록에 대해 **sub-second latency**를 내세우고, fork-aware streaming과 cursor-based resumption을 제공한다. SubQuery는 worker thread 사용 시 **최대 4배**의 인덱싱 속도 향상을 제시한다. NEAR는 공식 문서에서 Indexer와

Lake를 비교하며 **최소 3.8초, 평균 5-7초 대 최소 3.9초, 평균 6-8초** 반응시간과 **\$500+/월 vs \$20/월** 수준의 인프라 차이를 제시한다. Ethereum archive는 Geth path-based 모드가 **약 2TB 또는 6.5TB**, hash-based 모드가 **20TB 초과**, 동기화가 **약 2주 또는 수개월**까지 늘어날 수 있다고 공개한다. Solana는 slot이 통상 **약 400-600ms**이므로, 저지연 인덱서의 설계 목표 역시 “한 슬롯 내 또는 수 슬롯 이내”로 잡는 편이 현실적이다. ⁶

결론적으로, 실무 권장안은 다음과 같다. **원시 체인 이벤트를 변경 불가능한 append-only 로그로 먼저 보존하고, 그 위에 도메인별 projection을 별도로 구축하라. reorg-safe checkpoint와 block confirmation policy를 명시하라. query API는 별도 노드/서비스로 분리하라. PostgreSQL 파티셔닝과 인덱스 설계를 먼저 하고, 검색/랭킹/전문검색이 필요할 때만 Elasticsearch를 추가하라. 저지연이 핵심이면 streaming-first, 비용 효율과 재처리가 핵심이면 file/lake-first, 검증 가능성과 분산성이 핵심이면 full-node-first 또는 네트워크형 인덱싱을 택하라.** 이 보고서의 마지막 체크리스트는 바로 그 선택을 돕기 위해 구성했다. ⁷

인덱서의 정의와 역할, 그리고 왜 필요한가

인덱서는 체인의 블록 데이터를 **지속적으로 감시하고, 사전에 정의한 스키마에 맞게 정리한 뒤, 질의에 적합한 저장소에 적재하고, 효율적인 API로 제공하는 인프라 도구**다. Polkadot 문서는 이를 “특화된 데이터 접근 툴”로 정의하면서 블록과 트랜잭션을 계속 모니터링하고, 스키마에 따라 분류해 질의 가능한 DB와 API를 제공한다고 설명한다. SubQuery 역시 “블록체인에서 필요한 데이터만 빠르게 꺼내 전통적 DB에 저장”하는 과정을 블록체인 데이터 인덱싱의 본질로 설명한다. ⁸

성능 관점에서 인덱서는 필수에 가깝다. Polkadot 문서가 지적하듯 블록체인 데이터는 여러 블록에 흩어져 있고, 복잡한 교차 블록 질의와 집계는 수일 또는 수주가 걸릴 수 있으며, 직접적인 체인 질의는 dApp 성능과 응답성을 해친다. Solana 문서도 `getProgramAccounts` 같은 무거운 RPC 부하 때문에 validator가 네트워크를 따라가지 못할 수 있어, Geyser 플러그인으로 계정·슬롯·블록·트랜잭션을 외부 저장소로 내보내도록 했다고 밝힌다. NEAR 역시 복잡한 컨트랙트 상태 질의에서 RPC가 최적이지 않을 수 있다고 명시한다. ⁹

사용자 경험 관점에서 인덱서는 “지금 막 발생한 체인 이벤트”와 “역사 전체를 기반으로 한 가공 결과”를 동시에 만족시켜야 한다. SubQuery는 추가 레이어인 인덱서가 지연을 만들 수 있음을 전제로, 미확정(unconfirmed) 데이터의 실시간 인덱싱과 reorg 시 자동 롤백을 지원한다고 문서화한다. Solana는 `processed`, `confirmed`, `finalized` commitment를 공식적으로 구분하며, 가장 최신 상태는 롤백 가능하다고 설명한다. 이는 실무자가 UX를 위해 낮은 지연을 택할지, 정확성을 위해 높은 확정성을 택할지 정책적으로 정해야 함을 뜻한다. ¹⁰

데이터 접근성 관점에서는 원시 RPC가 충분치 않다. Ethereum은 JSON-RPC와 pub/sub, tracing API를 제공하지만 과거 상태 조회나 tracing은 archive node가 사실상 필요하다. Ethereum 공식 문서는 archive node가 지갑/탐색기/분석 서비스에 유용하다고 설명하고, Geth는 hash-based archive가 20TB를 초과할 수 있다고 밝힌다. 다시 말해 체인 프로토콜이 제공하는 표준 API는 “기본 원장 접근”에는 충분하지만, “풍부한 사용자 질의”에는 별도의 인덱싱 레이어가 필요하다. ¹¹

분산성 관점은 더 미묘하다. 인덱서는 한편으로는 탈중앙화를 보완한다. Ethereum 문서는 풀노드를 직접 운영하는 것이 가장 trustless하고 private하며 censorship resistant하다고 말한다. The Graph는 인덱서가 토큰을 스테이킹하고, 잘못된 데이터를 제공하거나 잘못 인덱싱하면 슬래시될 수 있도록 설계하여, 질의 레이어 자체를 네트워크 형태로 분산시킨다. 반면 NEAR의 비교 표는 과거 Lake Framework가 S3 기반이라 “Decentralized: No”였고, 직접 체인을 읽는 NEAR Indexer가 “Yes”라고 구분한다. 즉, 인덱서는 체인 위에 또 하나의 데이터 권력층이 될 수도 있고, 반대로 체인 접근 권한을 더 넓게 분산시키는 도구가 될 수도 있다. ¹²

실무적으로 이 긴장은 “무엇을 누가 검증하느냐”로 정리된다. 자체 풀노드 기반 인덱서는 높은 검증성과 높은 운영비를 갖고, 호스팅드 스트림 기반 인덱서는 빠른 개발과 낮은 초기비용을 주는 대신 외부 공급자 의존성을 높인다. 따라서 인덱서 전략은 단지 성능 문제가 아니라, 애플리케이션의 **위험 모델(risk model)**과 **신뢰 모델(trust model)**의 일부로 다뤄야 한다. ¹³

주요 구현 패턴과 아키텍처 구성요소

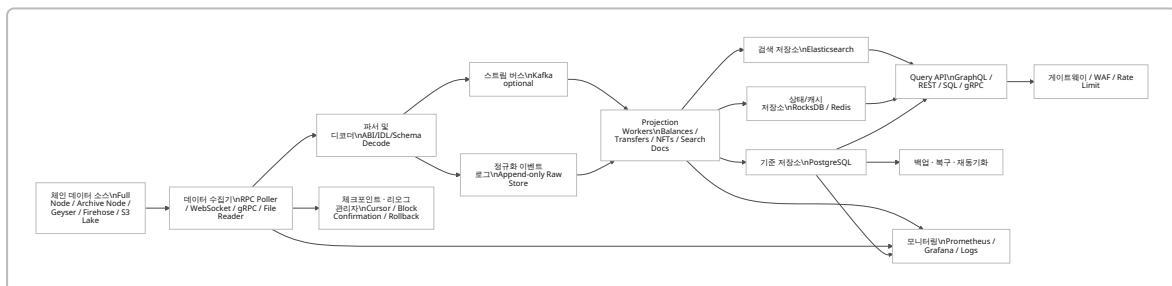
실무에서 가장 흔한 패턴은 **폴노드 기반 직접 수집**이다. 이 모델은 체인 노드의 JSON-RPC, WebSocket, tracing API 또는 노드 내부 프레임워크에서 직접 데이터를 읽는다. SubQuery EVM 문서는 인덱싱 엔드포인트로 **non-pruned archive node**를 권장하고, 공용 노드는 rate limit 때문에 속도에 악영향을 줄 수 있다고 경고한다. NEAR Indexer는 nearcore 노드를 돌리면서 블록 스트림을 직접 처리하는 구조다. Ethereum에서는 `eth_subscribe` 와 tracing API가 이 범주에 해당한다. 14

두 번째는 **이벤트 스트리밍 기반**이다. Solana Geyser는 validator 내부에서 계정·슬롯·블록·트랜잭션 업데이트를 PostgreSQL, NoSQL, Kafka 같은 외부 저장소로 밀어낼 수 있게 설계되었다. Solana 공식/Anza 문서는 바로 이 목적이 “validator를 무거운 RPC 질의에서 분리”하는 것이라고 설명한다. Yellowstone gRPC는 Geyser 위에 표준화된 스트리밍 인터페이스를 올려 슬롯·블록·트랜잭션·계정 알림을 제공한다. Ethereum 쪽에서는 WebSocket pub/sub가 보다 단순한 스트리밍 버전이며, The Graph/Firehose는 한 단계 더 나아가 스트리밍+파일 백업을 결합한다. 15

세 번째는 **트랜잭션/이벤트 스캔 기반**이다. 이 모델은 체인 전체를 훑되, handler filter·topic filter·address filter·dictionary를 이용해 “필요한 블록만” 선택적으로 처리한다. SubQuery는 dictionary를 사용하지 않을 때는 기본 batch-size만큼 모든 블록을 가져와 필터를 검사하지만, dictionary를 쓰면 GraphQL 질의로 “관련 이벤트/익스트린식이 있는 블록 높이만” 받아와 불필요한 스캔을 건너뛰는다고 설명한다. 이 패턴은 EVM logs, Substrate events/extrinsics, Blockscout catchup 등에서 매우 일반적이다. 16

네 번째는 **상태 스냅샷/데이터 레이크 기반**이다. NEAR Lake는 이벤트 로그를 JSON 파일로 AWS S3에 덤프했고, NEAR Lake Framework는 이를 읽어 간편한 self-hosted indexer를 만들도록 했다. 2026년 3월 24일부로 NEAR Lake는 신규 블록 인덱싱을 중단했고, NEAR는 Neardata 또는 Nearcore Indexer를 권장한다. Firehose도 같은 계열이지만 더 범용적이다. Firehose는 블록체인 데이터를 파일 기반·streaming-first로 추출·저장하고, sub-second live streaming, historical access, fork-aware streaming, cursor-based resumption을 제공한다. 이 패턴의 장점은 대량 재동기화와 병렬화가 쉽다는 점이다. 17

실제 아키텍처는 보통 아래와 같이 구성된다. 다음 도식은 The Graph, Solana Geyser, Firehose, SubQuery, Blockscout, NEAR ReadRPC의 공통 요소를 재구성한 실무형 참조 아키텍처다. 18



이 구조에서 핵심은 **수집 단계와 질의 단계의 분리**다. Graph Node는 전용 query node와 dedicated indexing node를 구분하고, Postgres shard를 여러 개 둘 수 있다. Blockscout도 indexer mode, API mode, webapp mode를 분리해 별도 확장이 가능하다. Solana는 아예 validator가 무거운 RPC 질의로 밀리지 않도록 Geyser로 외부화한다. 즉, “인덱서 성능”은 단일 프로세스의 속도보다 **파이프라인 분리**에서 더 많이 나온다. 19

재동기화와 리커버리는 별도 기능으로 설계해야 한다. Firehose는 cursor-based resumption과 fork-aware streaming을 제공하고, SubQuery는 block confirmation과 자동 롤백을 제공한다. Kafka는 consumer offset 재설정과 로그 재처리가 가능하며, 오래된 세그먼트를 retention 정책으로 관리한다. 이러한 기능은 “처음부터 다시 sync” 대신 “체크포인트 이후만 replay”하게 만들어 운영비를 크게 줄인다. 20

모니터링은 선택이 아니라 필수다. Graph Node는 Prometheus 메트릭 엔드포인트와 Grafana 예시를 제공하고, SubQuery는 인덱서 노드·쿼리 서비스 헬스체크와 CPU·메모리·디스크·외부 RPC·DB 용량을 모니터링하라고 권장한다. 운영 중 가장 흔한 장애는 “체인 헤드 추격 실패”, “DB 디스크 고갈”, “RPC rate limit”, “reorg 후 projection 불일치”인데, 네 가지 모두 모니터링 부재에서 악화된다. ²¹

데이터 모델링과 비기능 요구사항

좋은 인덱서의 데이터 모델은 원시 블록 구조를 그대로 복사하지 않는다. 대신 **체인 공통 엔터티와 도메인 엔터티를 분리**한다. 예를 들어 공통 계층에는 Block, Transaction, Log/Event, StateChange, Checkpoint가 들어가고, 도메인 계층에는 TokenTransfer, NFTTransfer, PoolSwap, AccountBalance, OrderBookFill 같은 projection이 들어간다. SubQuery 문서는 GraphQL schema가 데이터 모양을 결정한다고 명시하고, 각 엔터티에 id, 관계, 보조 인덱스, JSON 필드, reverse lookup을 설계할 수 있다고 설명한다. PostgreSQL은 대용량 파티셔닝과 jsonb 인덱싱을 제공하므로, “정형 컬럼 + 제한적 JSON 확장” 모델이 보통 가장 다루기 쉽다. ²²

아래 스키마 예시는 EVM·Substrate·NEAR·Solana에 공통적으로 적용 가능한 **정규화된 블록체인 인덱서의 최소 코어 스키마**를 GraphQL 형태로 재구성한 것이다. 실무에서는 이 위에 token/NFT/DEX projection을 별도 모듈로 엮는 편이 유지보수와 마이그레이션에 유리하다. 이 구조는 SubQuery와 The Graph의 entity 중심 모델, Blockscout의 다층 fetcher 구조, NEAR ReadRPC의 state-indexer / tx-indexer 분리를 반영한 것이다. ²³

```
type Block @entity {
  id: ID!
  chainId: String! @index
  height: BigInt! @index(unique: true)
  hash: Bytes! @index(unique: true)
  parentHash: Bytes! @index
  timestamp: Date! @index
  status: String! @index # processed / confirmed / finalized
}

type Transaction @entity {
  id: ID!
  block: Block! @index
  hash: Bytes! @index(unique: true)
  from: Bytes @index
  to: Bytes @index
  nonce: BigInt
  success: Boolean! @index
  gasUsed: BigInt
  feePaid: BigInt
}

type Event @entity {
  id: ID!
  tx: Transaction! @index
  block: Block! @index
  contractOrProgram: Bytes @index
}
```

```

topic0: String @index
eventName: String @index
logIndex: Int @index
payload: JSON
}

type StateChange @entity {
  id: ID!
  block: Block! @index
  account: Bytes! @index
  key: String! @index
  value: JSON
}

type MaterializedBalance @entity {
  id: ID!
  account: Bytes! @index
  asset: Bytes! @index
  balance: BigInt! @index
  asOfBlock: BigInt! @index
}

type Checkpoint @entity {
  id: ID!
  worker: String! @index(unique: true)
  lastProcessedHeight: BigInt! @index
  lastFinalizedHeight: BigInt! @index
  cursor: String
  updatedAt: Date! @index
}

```

비기능 요구사항은 설계 초기에 명시해야 한다. 아래 표는 인덱서에서 반드시 정의해야 할 요구사항과 권장 설계 전술을 정리한 것이다. ²⁴

요구사항	실무 의미	권장 설계 전술	대표 지표	주요 근거
일관성·정합성	reorg, 미확정 블록, 중복 수신에도 결과가 틀리지 않아야 함	block confirmation 정책, append-only raw log, idempotent upsert, cursor 저장, rollback/replay	rollback 성공률, data drift 건수	²⁵
지연시간	UX에 보이는 최신성	소스별 commitment 분리, hot projection 우선, cache/streaming 사용	head lag, p95 ingest latency	²⁶
처리량	고TPS 체인/대규모 백필 대응	filter/dictionary, parallel worker, Kafka partitioning, bulk write	blocks/s, tx/s, rows/s	²⁷
확장성	체인 수·프로젝션 수·질 의량 증가에 대응	API/Indexer 분리, query node 분리, DB sharding, table partitioning	shard별 QPS, DB wait time	²⁸

요구사항	실무 의미	권장 설계 기술	대표 지표	주요 근거
보안	데이터 위조·DoS·오염 방지	서명된 receipts, WAF/API gateway, query complexity 제한, 최소 포트 노출	invalid proof 비율, rate-limit hit	29
프라이버시	오프체인 보강 데이터와 운영 로그의 노출 최소화	query receipt/로그 분리, TLS, 접근제어, 최소 정보 보관	PII 저장 항목 수, 암호화 범위	29
운영성	백업·재동기화·장애 복구	raw log 보존, snapshot 백업, 재처리 파이프라인, Grafana/Prometheus	RPO, RTO, replay 시간	30
버전관리·마이그레이션	스키마 변경 때 서비스 중단 최소화	schema migration flag, blue/green projection, versioned API	migration 시간, rollback 가능 여부	31

정합성 요구사항은 특히 체인별로 다르다. Solana는 `processed`가 가장 최신이지만 롤백될 수 있고, `finalized`가 가장 보수적이다. SubQuery는 EVM 프로젝트에서 기본 block confirmations를 200으로 두고 자동 롤백을 수행한다. Firehose는 fork-aware streaming과 cursor-based resumption을 제공한다. 따라서 “이 인덱서의 질의 결과가 어느 확정 수준을 반영하는가”를 API 계약서에 반드시 명시해야 한다. ³²

처리량과 확장성을 높이는 핵심은 **데이터를 덜 읽고, 덜 쓰고, 더 짧게 쓰는 것이다**. SubQuery는 mapping filter와 dictionary가 처리 블록 수를 크게 줄인다고 설명하고, worker thread와 store cache 조정으로 최대 4배 가속을 제시한다. PostgreSQL은 파티셔닝으로 hot partition이 메모리에 남게 해 질의 성능을 개선하고, `DROP/DETACH PARTITION`으로 bulk delete를 빠르게 수행할 수 있다. 실무에서는 `height`, `timestamp`, `contract/program`, `account` 기준의 복합 인덱스와 월별·높이대별 파티셔닝이 가장 흔하다. ³³

검색과 분석 요구가 강하면 별도 검색 엔진을 붙이는 편이 낫다. Elasticsearch는 JSON document 중심 저장과 shard 기반 수평확장을 제공하며, metric/bucket/pipeline aggregation을 공식적으로 지원한다. 반면 기존 저장소 역할은 여전히 PostgreSQL이 더 낫다. 따라서 “정합성의 원본(DB of record)은 Postgres, 검색과 전문 집계는 Elasticsearch”의 이중 구조가 일반적으로 안전하다. ³⁴

운영 측면에서는 스키마 변경을 인덱스 리플레이와 분리해야 한다. NEAR Indexer for Explorer 저장소는 공개 SQL 스키마가 진화할 수 있으므로 release note를 보라고 경고하고, 일부 migration은 인덱스 생성에 `CONCURRENTLY` 옵션을 직접 써서 수동 적용하라고 조언한다. SubQuery는 `--allow-schema-migration` 플래그로 자동 스키마 마이그레이션을 지원한다. 즉, **스키마 버전업 = 곧바로 전체 재색인**이어서는 안 되며, projection 버전과 백필 전략을 분리해야 한다. ³¹

권장 기술스택과 오픈소스 비교

아래 비교표는 2026-06-01 기준으로 실무에서 자주 결합되는 인덱스 프레임워크와 저장/스트리밍 기술의 역할을 요약한 것이다. 표의 평가는 문서·레포지토리의 공식 기능 설명을 바탕으로 실무 관점에서 재구성한 것이다. ³⁵

기술	주 용도	장점	단점	적합한 상황	주요 근거
The Graph / Graph Node	Subgraph 기반 인덱싱 · GraphQL 질의	분산형 indexer 네트워크, query node 분리, Postgres shard 지원, Prometheus/ Grafana 운영성	GraphQL/ entity 모델 중심이라 매우 자유로운 변환 작업에는 제약, Postgres 의존성 큼	dApp용 공개 API, 표준 이벤트 중심 도메인	36
Firehose / Substreams	대규모 병렬 추출·변환	sub-second live streaming, fork-aware streaming, cursor resume, 병렬 실행, 일부 subgraph 100x+ sync 가속	도입 복잡도와 학습 비용이 높고, 별도 저장·서빙 계층 설계 필요	대규모 백필, 멀티프로젝션, 고성능 ETL	37
SubQuery	멀티체인 인덱서 SDK + GraphQL	EVM · Substrate · NEAR 등 폭넓은 지원, filter/dictionary, worker thread, schema migration, query protection	프레임워크 기능이 많아 구성 복잡도 증가, 고급 운영은 여전히 Postgres/ RPC 품질에 좌우	멀티체인, 빠른 개발, 자체 호스팅 또는 네트워크 배포	38
Blockscout	EVM 탐색기/인덱서	realtime + catchup 이중 인덱서, 다수 secondary fetcher, API/UI/Indexer 분리, 체인 특화 fetcher 존재	explorer 성격이 강해 범용 데이터 웨어하우스에는 추가 가공 필요	탐색기, API, verification, 멀티 EVM 운영	39
Kafka	스트림 버스, 재처리, 역압 제어	partition 기반 병렬성, offset replay, retention/ compaction, replication · idempotence 옵션	운영 복잡도, broker/ partition 설계 필요	high-volume event bus, fan-out 파이프라인	40
PostgreSQL	기존 저장소, GraphQL 백엔드	강한 SQL, 파티셔닝, jsonb, 보조 인덱스, 운영 성숙도 높음	쓰기 폭주·대규모 전문검색은 약점, shard는 운영 난이도 증가	대부분의 인덱서의 system of record	41
RocksDB	로컬 상태 저장·고속 KV	LSM 기반 고성능, SSD 친화적, 임베디드, 저지연	운영형 질의엔 불편, 직접 API/관리 기능이 적음	state store, cache, checkpoint	42

기술	주 용도	장점	단점	적합한 상황	주요 근거
Elasticsearch	검색·분석·전문검색	shard 기반 확장, JSON documents, aggregations, full-text search	기존 저장소로 쓰기엔 적합성 관리 부담, 스키마 튜닝 필요	탐색기 검색, 로그 분석, 랭킹·검색 UI	34

실무 추천 조합은 워크로드에 따라 다르다. **고성능 분석형**은 Firehose/Substreams + Kafka + PostgreSQL + Elasticsearch 조합이 강하고, **dApp API형**은 Graph Node 또는 SubQuery + PostgreSQL + Redis 캐시가 단순하다. **탐색기형**은 Blockscout이 이미 secondary fetcher와 verification 마이크로서비스를 갖추고 있어 구현 비용이 가장 낮다. **초저지연 Solana형**은 Geyser/Yellowstone + Kafka 또는 직접 Postgres + 로컬 KV 캐시가 적합하다. 43

선택 기준을 한 문장으로 정리하면 이렇다. “**백필과 재처리가 핵심이면 streaming/file-first, 운영 단순성이 핵심이면 framework + Postgres-first, 검색 UX가 핵심이면 search engine**을 보조 인덱스로 추가, 탐색기면 Blockscout, 검증 가능한 분산 질의면 The Graph”가 가장 실용적이다. 44

실제 사례 분석

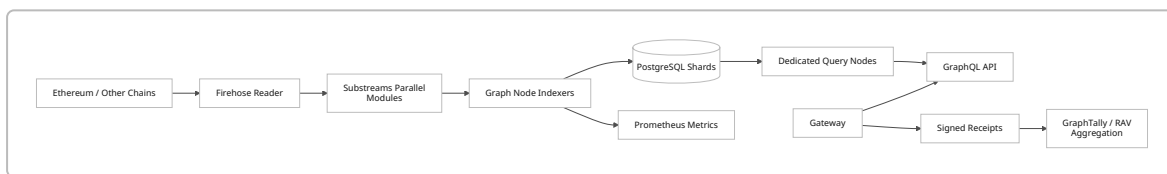
아래 비교표는 대표 다섯 사례를 한눈에 보는 용도다. 성능 수치가 프로젝트마다 다른 형식으로 공개되어 있어, **공식 문서가 직접 제시한 수치만 우선 반영**했고, 나머지는 공개 설정값·저장소 요구치·지연 예산으로 비교했다. 일부 통합 TPS/CPU 수치는 공식 문서에 없어서 비워 두었다. 45

사례	데이터 원천	핵심 패턴	공개 수치 또는 설정값	강점	한계	주요 근거
The Graph	Graph Node + Firehose/Substreams	entity projection + sharding + query node	일부 subgraph sync 100x+, Prometheus endpoint, Postgres sharding	분산형 indexer 네트워킹, 표준 GraphQL	Postgres 의존, 대규모 변환은 Substreams 설계 필요	46
NEAR	nearcore Indexer / 과거 Lake / ReadRPC	full-node stream + state/tx indexer 분리	3.8s min / 5-7s avg, Lake 6-8s avg, \$500+/mo vs \$20/mo, Explorer mainnet 3TB(문서 시점)	체인 직접 읽기, 상태 변화 모델 명확	Lake는 2026-03-24 종단, 일부 공개 수치는 역사적	47
Polkadot/Substrate + SubQuery	RPC + dictionary + filters	selected block scan + GraphQL	worker threads 최대 4배, 기본 batch-size 100, default block confirmations 200(EVM)	멀티체인, filter/dictionary 최적화	RPC 품질에 민감, 운영 지식 필요	48

사례	데이터 원천	핵심 패턴	공개 수치 또는 설정값	강점	한계	주요 근거
Solana	Geyser / Yellowstone / WebSocket	validator- side streaming	slot 400- 600ms, plugin 예시 threads=20 batch_size=20	초저지연, validator/ RPC 분리	고TPS·대용 량, 운영 리소 스 큼	49
Ethereum + Blockscout	JSON-RPC / eth_subscribe / tracing / archive	realtime + catchup + secondary fetchers	path archive 약 2TB 또는 6.5TB, hash archive 20TB+, sync 약 2주 또 는 수개월	explorer/ API 운영성, historical/ tracing 강 함	스토리지·컴 퓨트 비용 큼	50

The Graph는 인덱서 네트워크 자체를 프로토콜로 만든 사례다. 인덱서는 GRT를 스테이킹하고 query fee와 indexing reward를 받으며, 잘못된 데이터 제공이나 잘못된 인덱싱 시 슬래시될 수 있다. 운영 아키텍처에서는 Graph Node가 전용 query node를 둘 수 있고, Postgres shard를 여러 개로 수평 분리할 수 있으며, Firehose/Substreams를 결합하면 일부 서브그래프의 sync가 100배 이상 빨라질 수 있다고 문서화되어 있다. 질의 과금은 GraphTally의 signed receipt와 RAV(redeemable aggregate voucher)로 처리되어, 데이터 평면과 결제 평면이 분리된다. 51

다음 도식은 The Graph의 공식 문서와 리포지토리를 바탕으로 재구성한 운영형 배치다. 52

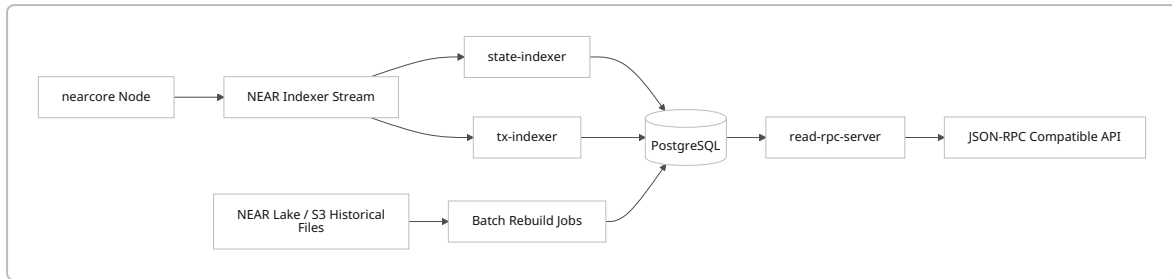


실무 포인트는 두 가지다. 첫째, **entity projection의 강점**이다. 스키마가 일찍 안정되면 개발 속도가 빠르다. 둘째, **대규모 백필은 Substreams로 앞단을 병렬화**해야 한다는 점이다. 순수 Graph Node만으로도 충분한 워크로드가 있지만, 대형 DEX·시계열 집계·체인 전체 분석은 Firehose/Substreams 계층이 사실상 유리하다. 53

NEAR는 “직접 체인을 읽는 인덱서”와 “오브젝트 스토리지 기반 Lake”를 모두 운영해 본 사례라 설계 트레이드오프가 매우 명확하다. NEAR 공식 문서는 RPC가 복잡한 상태 질의에 최적이지 아닐 수 있다고 하고, NEAR Indexer는 블록이 추가될 때마다 nearcore가 제공하는 스트림을 처리한다. 반면 과거 NEAR Lake는 이벤트 로그를 S3 JSON으로 저장했고, 2026년 3월 24일부로 신규 블록 인덱싱을 중단했다. 문서 비교표는 Indexer가 Lake보다 지연이 더 낮고 분산적이지만, 인프라 비용과 유지보수 난도가 높다고 설명한다. 54

NEAR ReadRPC 리포지토리는 이 철학을 더 구체화한다. **rpc-server**는 Postgres/S3/실시간 RPC를 조합해 읽기 전용 JSON-RPC를 제공하고, **state-indexer**는 상태변화를 저장하며, **tx-indexer**는 transactions/receipts/execution outcomes를 분리 적재한다. 이는 “원시 체인 입수”와 “API 제공”을 분리하는 전형적인 CQRS 구조에 가깝다. 55

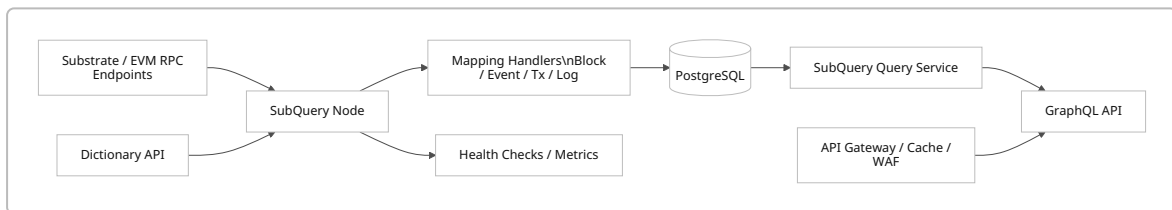
다음 도식은 NEAR Indexer / ReadRPC 계열 설계를 요약한 것이다. 56



NEAR 사례의 실무 교훈은 명확하다. **간편 개발과 저비용 백필이 필요하면 file/lake가 유리하고, 분산성과 낮은 지연이 필요하면 full-node 기반이 낫다.** 또한 문서에 따르면 Explorer용 Postgres 스토리지가 2022년 시점 mainnet에서 이미 3TB였으므로, 장기 운영 시 스토리지 계획을 미리 세워야 한다. 57

Polkadot/Substrate + SubQuery는 “선택적 스캔”의 모범 사례다. Polkadot 문서는 인덱서가 블록체인 데이터 접근 문제를 해결하기 위해 블록과 트랜잭션을 지속적으로 모니터링하고 GraphQL API를 제공한다고 설명한다. SubQuery는 여기에 dictionary, mapping filter, worker thread, block confirmation, schema migration, multi-chain manifest를 엮었다. 특히 dictionary는 relevant block height만 받아와 불필요한 블록 처리량을 줄이고, filter는 event/extrinsic/transaction/topic 수준에서 입력량을 줄인다. 58

다음 도식은 SubQuery 인덱서의 전형적 운영 형태를 재구성한 것이다. 59

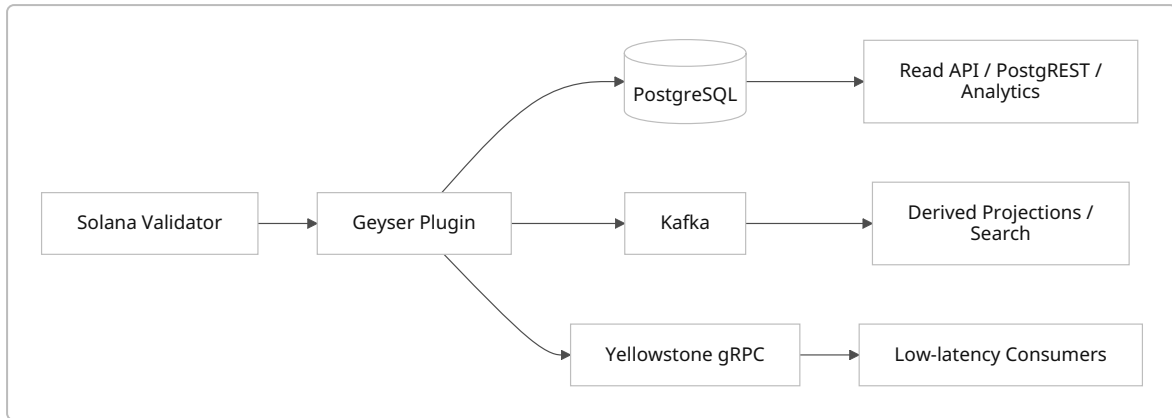


실무적으로 SubQuery의 강점은 **프로젝트 구조가 명확하고 멀티체인 확장이 자연스럽다**는 점이다. `project.ts`에서 data source, startBlock, ABI, handler, filter를 선언하고, `schema.graphql`로 projection을 정의하며, query service가 GraphQL을 노출한다. 문서에서 worker thread는 최대 4배 가속, public RPC rate limiting 경고, query complexity 제한, API gateway/WAF 권장을 모두 제공하므로, 프레임워크와 운영 가이드를 함께 준다는 장점이 있다. 60

Solana는 가장 저지연이 강조되는 체인 중 하나이고, 따라서 인덱서 구조 역시 “validator 옆에서 바로 뽑아내는 방식”으로 발달했다. Anza 문서는 validator가 무거운 RPC 질의에 밀릴 수 있어 Geyser plugin으로 accounts, slots, blocks, transactions를 PostgreSQL, NoSQL, Kafka로 전송한다고 설명한다. Yellowstone gRPC는 이 Geyser 위에 slots, blocks, transactions, account updates, 심지어 deshred pre-execution transactions까지 전달하는 표준 gRPC 계층을 엮는다. 솔라나 RPC 문서는 shared public endpoint를 production 용도로 권장하지 않으며 429/403이 발생할 수 있다고 분명히 적고 있다. 61

Postgres reference plugin의 README는 `threads: 20`, `batch_size: 20` 같은 설정 예시를 제공하며, thread 수를 높이면 DB throughput이 대개 더 좋아진다고 설명한다. 이 값이 곧 권장 수치라는 뜻은 아니지만, Solana 인덱서가 보통 **멀티스레드 + 배치 적재 + 외부화된 상태 저장소**를 전제로 설계된다는 점은 분명하다. 또한 Solana slot은 보통 400~600ms이므로, “한 슬롯 이내 알림”은 곧 상당한 고성능 요구를 의미한다. 62

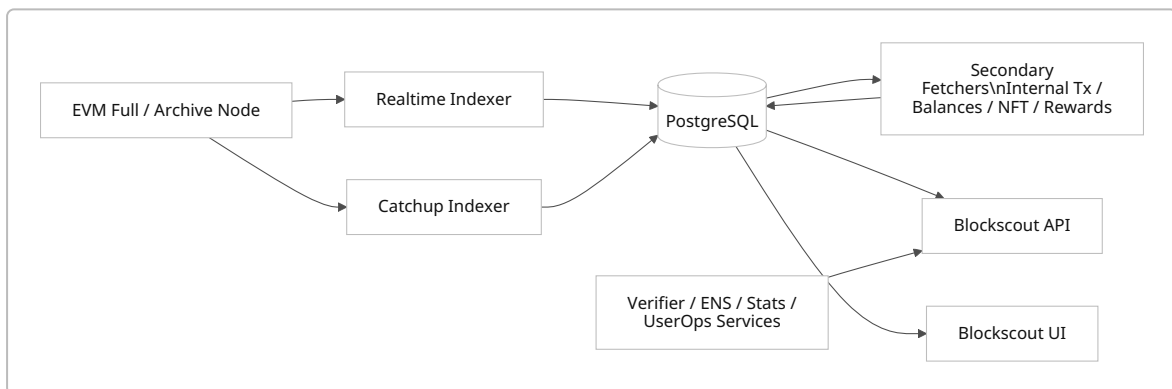
다음 도식은 Solana Geyser / Yellowstone 계열 구조를 요약한 것이다. 61



Solana 사례의 교훈은 “노드와 질의의 분리”가 다른 체인보다 더 중요하다는 점이다. 계정 모델이 key-value 중심이고 슬롯 주기가 매우 짧기 때문에, 탐색기/지갑/API를 validator 위에 직접 붙이는 대신 Geyser나 gRPC 스트림으로 외부 질의 계층을 구성하는 편이 안전하다. 다만 공식 문서에는 end-to-end 처리량 수치가 제한적으로만 공개되어 있으므로, 실제 capacity planning은 자신의 워크로드로 benchmark해야 한다는 한계가 있다. 이 단락의 마지막 판단은 공식 문서 구조를 기반으로 한 실무적 추론이다. 63

Ethereum + Blockscout는 “폴노드/아카이브/탐색기 인덱서”가 만나는 전형 사례다. Ethereum은 JSON-RPC, WebSocket pub/sub, tracing API, archive/full node를 제공한다. Geth는 path-based archive 도입으로 historical state 보관 비용을 크게 줄였지만 여전히 2TB 또는 6.5TB, legacy hash-based는 20TB 초과와 수개월 sync가 가능하다고 설명한다. Blockscout는 이 원천 위에 realtime importer와 catchup importer, 그리고 internal transactions, pending tx, dropped/replaced tx, contract bytecodes, balances, NFT instances, chain-specific fetcher 등 다층 secondary fetcher를 올린다. 64

다음 도식은 Blockscout 중심 EVM 인덱서 구조를 재구성한 것이다. 65



실무 포인트는 세 가지다. 첫째, **탐색기용 인덱서는 원시 블록 적재만으로 끝나지 않는다**. 내부 트랜잭션, 컨트랙트 검증, ENS/서명 해석, ERC-4337 user operation 같은 “파생 데이터”가 훨씬 많다. 둘째, **mode separation**이 중요하다. Blockscout는 indexer/API/UI를 분리해서 같은 DB를 공유할 수 있다. 셋째, **historical query 비용**이 매우 커진다. explorer·analytics·wallet 벤더가 archive node를 선호하는 이유와 부담이 동시에 드러나는 사례다. 66

보안, 공격면, 운영 요구사항, 그리고 비용 추정 모델

인덱서의 보안은 체인 보안과 다르다. 체인이 합의를 보장하더라도, 인덱서는 **잘못된 소스를 읽거나, 미확정 데이터를 조기 반영하거나, 중복·재전송을 잘못 처리하거나, 질의 API가 남용되는 순간** 쉽게 오염된다. 대표 공격면은 데이터 위조/오염, replay와 reorg, DoS, query 폭주, off-chain enrichment 오염, 그리고 운영자 실수에 의한 스키마 불일치다. The Graph는 signed receipt와 on-chain redemption으로 query payment 무결성을 보완하고, SubQuery는

gateway/WAF와 rate limit, 쿼리 복잡도 제한, 최소 포트 노출을 권장한다. Solana와 Ethereum은 각각 commitment/finality와 subscription 모델을 제공해 명확한 데이터 정책을 분리하도록 한다. 67

데이터 위조와 잘못된 결과 제공에 대한 첫 번째 방어선은 “검증 가능한 소스”다. Ethereum은 풀노드가 가장 trustless하고 censorship resistant한 접근이라고 설명한다. The Graph는 인덱서 부정행위에 대해 슬래싱을 명시하고 query receipt를 암호학적으로 검증한다. 따라서 고신뢰가 필요한 서비스는 최소한 하나 이상의 자체 검증 소스(full/archive/light client, protocol-integrated receipt verification)를 가져가는 편이 좋다. 68

replay와 reorg에 대한 방어선은 “append-only raw log + idempotent projection + checkpoint + rollback” 조합이다. Firehose는 fork-aware streaming과 cursor-based resume를 제공하고, SubQuery는 reorg 시 자동 롤백을 제공한다. Solana는 processed / confirmed / finalized commitment를 구분하여 애초에 API 계약서 수준에서 재구성 허용 범위를 드러낸다. 실무 권고는, 모든 projection row에 as_of_block 또는 finalized_at 을 남기고, raw log를 삭제하지 않는 것이다. 69

DoS와 query abuse는 인덱서가 체인보다 먼저 죽는 전형적 원인이다. Solana는 shared public RPC가 production용이 아니며 429 와 403 이 발생할 수 있다고 공식 문서에 명시한다. SubQuery는 query service 앞단에 API gateway·WAF·rate limit·TTL 캐시·필수 포트만 노출할 것을 권장한다. Graph Node와 Blockscout도 API와 query plane을 분리해 별도 확장 가능하므로, 공용 인터넷에는 query plane만 노출하는 것이 정석이다. 70

운영 요구사항은 네 가지로 정리된다. **백업**은 원시 로그와 projection DB를 나눠 수행해야 한다. **마이그레이션**은 online index build와 blue/green projection을 고려해야 한다. **버전관리**는 schema version과 API version을 분리해야 한다. **모니터링**은 ingestion lag, DB disk, external RPC health, query p95, replay duration, proof validation error를 포함해야 한다. NEAR는 release note 추적과 수동 migration 검토를 권장하고, SubQuery는 자동 schema migration과 헬스체크 가이드를 제공하며, Graph Node는 Prometheus/Grafana를 공식 예시로 제공한다. 71

비용은 보통 아래 공식으로 추정하는 것이 가장 현실적이다. 이 식은 공급자별 단가가 달라도 동일하게 적용된다. 다만 아래의 예시 수치 중 **스토리지 단가와 일부 요청 단가만 공식 가격을 직접 사용**했고, compute와 egress는 워크로드·리전·예약형태에 따라 크게 달라져 변수로 두었다. 72

월 총비용 =
노드 컴퓨트(풀노드/아카이브/인덱서/API)
+ DB 스토리지(하트)
+ 오브젝트 스토리지(백업/레이크)
+ 스트림 버스 비용(Kafka 등)
+ 네트워크 egress
+ 모니터링/로그 저장
+ 운영 인력비

공식 단가와 공개 저장소 요구치를 결합하면 스토리지 비용의 하한은 꽤 정확히 산정할 수 있다. AWS gp3는 **\$0.08/GB-month**이고, Geth path-based archive는 약 **2TB** 또는 **6.5TB**, hash-based archive는 **20TB** 초과가 될 수 있다. 이를 단순 계산하면 gp3 기준으로 2TB는 약 **\$163.84/월**, 6.5TB는 약 **\$532.48/월**, 20TB는 약 **\$1,638.40/월**의 스토리지 비용만 발생한다. 이는 compute·snapshot·백업·egress를 제외한 값이므로, Ethereum archive 기반 인덱서는 저장소만으로도 중형 SaaS 한 대보다 비싸질 수 있다. 73

오브젝트 스토리지 비용은 생각보다 낮지만 요청 수와 egress가 붙으면 빠르게 커진다. AWS 공식 문서 예시는 S3 Standard가 **\$0.023/GB-month**, PUT/COPY/POST/LIST가 **\$0.005/1,000 requests**라고 제시한다. 따라서 1TB

백업 스냅샷을 S3 Standard에만 보관하면 저장료는 약 **\$23.55/월** 수준이지만, lake형 인덱서가 아주 작은 파일을 많이 생성하거나, 외부 소비자가 대량 다운로드를 수행하면 요청·전송 비용이 저장료보다 커질 수 있다. ⁷⁴

실무용 예산 모델은 다음 세 단계로 나누는 것이 유용하다. 이 표의 compute는 공급자 중립적 추정 범주이며, 공식 문서가 공개한 스펙과 스토리지 단가를 바탕으로 작성했다. 구체적인 월 과금액은 리전·예약형태·managed DB 사용 여부에 따라 달라진다. ⁷⁵

규모	전형적 워크로드	인프라 형태	비용 드라이버	실무 해석
경량	특정 컨트랙트/프로그램 이벤트만 수집	private RPC + indexer app + Postgres 수백 GB	컴퓨터보다 DB 저장·쿼리	빠른 개발과 저비용에 적합
중간	멀티프로토콜 API, explorer-lite, search 포함	full node 또는 managed stream + Kafka optional + Postgres 1-3TB + ES	DB IOPS, 인덱스 수, query traffic	가장 흔한 프로덕션 형태
중량	Ethereum explorer/analytics, Solana low-latency infra	archive/full node cluster + streaming + 대형 DB + search + backup	archive storage, sustained compute, egress	자체 인프라 팀 없으면 운영 난도 매우 높음

비용 최적화의 핵심은 세 가지다. **첫째, raw와 projection을 분리해 projection을 언제든 재생성 가능하게 만들어야 한다.** 그러면 projection 백업 주기를 공격적으로 줄일 수 있다. **둘째, 필요한 블록만 읽어야 한다.** dictionary, topic/address filter, startBlock 최적화가 곧 비용 절감이다. **셋째, archive node를 무조건 직접 운영하지 말아야 한다.** 개발·초기 제품 단계에서는 private archived RPC 또는 lake/firehose 공급자를 쓰고, 트래픽이 비용을 역전시키는 시점에만 self-hosting을 고려하는 편이 일반적으로 유리하다. ⁷⁶

설계 권장사항과 체크리스트

권장 설계를 한 문장으로 요약하면, “**검증 가능한 소스에서 raw append-only 이벤트를 보존하고, idempotent projection을 별도로 만들며, query plane을 독립 확장하고, reorg 정책을 API 계약으로 노출하라**”다. 이 원칙은 The Graph, NEAR ReadRPC, Solana Geyser, SubQuery, Blockscout가 서로 다른 기술 스택을 쓰면서도 공통적으로 보여 주는 패턴이다. ⁷⁷

실무 설계 권장사항은 다음과 같다.

첫째, raw log를 절대 authoritative source로 두고 projection은 disposable하게 만들라. Firehose의 cursor와 compressed block file, Kafka의 offset replay, Blockscout의 temporary fetcher 철학이 이 접근을 뒷받침한다. ⁷⁸

둘째, API 응답에서 확정 수준을 분리하라. processed/confirmed/finalized, pending/final, latest/finalized처럼 API 스펙에 노출해야 하며, 그렇지 않으면 UX팀과 데이터팀이 서로 다른 숫자를 보게 된다. SubQuery block confirmations, Solana commitment, GraphTally finality handling은 모두 같은 문제를 겨냥한다. ⁷⁹

셋째, 저장소는 역할별로 나뉘라. Postgres는 기준 저장소, RocksDB/Redis는 상태·캐시, Elasticsearch는 검색·집계, Kafka는 스트림 버스로 쓰는 편이 안정적이다. 하나의 시스템에 모든 요구를 억지로 몰아넣으면 성능과 운영성이 동시에 악화된다. ⁸⁰

넷째, **node plane**과 **query plane**을 분리하라. Graph Node query node, Blockscout indexer/API/UI separation, Solana Geyser 외부화는 모두 이 원칙의 사례다. 질의 트래픽이 ingest를 방해하는 구조는 장기적으로 반드시 장애를 만든다. ⁸¹

다섯째, **schema migration**을 제품 릴리스와 분리하라. NEAR와 SubQuery 문서 모두 migration 운영을 별도로 다룬다. projection 버전업이 필요하면 현재 버전과 다음 버전을 병행 운영하고, 백필 완료 후 스위칭하는 식이 가장 안전하다. ³¹

여섯째, **관측 가능성(observability)**을 처음부터 넣어라. 최소한 `head lag`, `finalized lag`, `DB connection wait`, `RPC error rate`, `replay duration`, `query p95`, `disk free`, `queue depth` 는 있어야 한다. Graph Node와 SubQuery가 공식적으로 Prometheus/health monitoring를 강조하는 이유가 여기에 있다. ²¹

아래 체크리스트는 신규 인덱서 프로젝트의 설계 리뷰 때 그대로 사용할 수 있는 형태로 정리했다. 각 항목에서 “예”가 아니면 운영 리스크가 높다고 보는 편이 맞다. ⁸²

- 데이터 원천이 명확한가. 자체 폴노드, archive RPC, Firehose, Geyser, lake 중 무엇을 기준으로 삼는가. ⁸³
- API가 어떤 확정 수준을 반환하는지 명시했는가. `latest` 와 `finalized` 를 구분하는가. ⁸⁴
- raw append-only 로그를 보존하는가. projection만 저장하고 있지는 않은가. ⁸⁵
- reorg/replay 시나리오를 테스트했는가. checkpoint와 rollback 로직이 있는가. ⁸⁶
- 필요한 블록만 읽도록 filter/dictionary/startBlock를 최적화했는가. ⁸⁷
- 기존 저장소와 검색 저장소를 분리했는가. Postgres와 Elasticsearch의 역할을 혼동하지 않는가. ⁸⁸
- query plane만 외부에 공개하고, WAF/rate limit/query complexity 제한을 두었는가. ⁸⁹
- DB 파티셔닝과 주요 보조 인덱스(`height`, `timestamp`, `contract`, `account`)를 설계했는가. ⁴¹
- schema migration 전략이 있는가. online migration, dual-write, backfill, rollback 중 무엇을 쓸지 정했는가. ³¹
- 모니터링 항목과 SLO를 정의했는가. head lag와 query p95를 추적하는가. ²¹
- 비용 모델에서 storage와 egress를 각각 계산했는가. archive node가 정말 필요한지 검증했는가. ⁹⁰

마지막으로, 이 보고서의 가장 중요한 실무 결론은 “인덱서는 단순 캐시가 아니라, 체인과 사용자 사이의 데이터 운영체제”라는 점이다. 성능, UX, 정확성, 보안, 비용, 분산성은 서로 교환관계에 있고, 그 균형점을 정하는 것이 바로 인덱서 설계다. 체인마다 최적해는 다르지만, 공식 문서와 오픈소스가 반복해서 보여 주는 공통 답은 이미 분명하다. **raw**를 보존하고, **projection**을 나누고, **확정성을 계약으로 만들고, query plane**을 격리하라. 그 네 가지를 지키면 대부분의 실패를 피할 수 있다. ⁹¹

Open questions / limitations

이 보고서는 2026-06-01 기준의 공식 문서·오픈소스 리포지토리·기술 문서를 우선 사용했다. 다만 일부 프로젝트는 **통합 처리량, CPU/메모리 사용량, p99 질의 지연** 같은 운영 수치를 공식 문서에서 상세히 공개하지 않았기 때문에, 해당 항목은 공개된 구조·설정값·스토리지 요구치 중심으로 비교했다. 특히 Solana와 Blockscout의 end-to-end 처리량은 워크로드 의존성이 커서 자체 벤치마크가 필요하다. 또한 비용 표는 공급자·리전·예약 정책에 따라 달라지므로, 본문 수치 중 스토리지 비용은 공식 단가 기반의 하한선으로 이해하는 것이 적절하다. ⁹²

¹ ² ⁸ ⁹ ⁵⁸ <https://docs.polkadot.com/parachains/integrations/indexers/>
<https://docs.polkadot.com/parachains/integrations/indexers/>

³ ¹⁰ ¹⁴ ²⁴ ²⁵ ⁷⁹ ⁸² <https://subquery.network/doc/indexer/build/manifest/chain-specific/ethereum.html>
<https://subquery.network/doc/indexer/build/manifest/chain-specific/ethereum.html>

4 22 23 60 <https://subquery.network/doc/indexer/build/introduction.html>
<https://subquery.network/doc/indexer/build/introduction.html>

5 19 21 28 30 52 81 <https://thegraph.com/docs/en/indexing/tooling/graph-node/>
<https://thegraph.com/docs/en/indexing/tooling/graph-node/>

6 45 46 53 <https://thegraph.com/blog/substreams-parallel-processing/>
<https://thegraph.com/blog/substreams-parallel-processing/>

7 41 80 88 <https://www.postgresql.org/docs/current/ddl-partitioning.html>
<https://www.postgresql.org/docs/current/ddl-partitioning.html>

11 <https://ethereum.org/developers/docs/apis/json-rpc/>
<https://ethereum.org/developers/docs/apis/json-rpc/>

12 13 68 83 <https://ethereum.org/developers/docs/nodes-and-clients/light-clients/>
<https://ethereum.org/developers/docs/nodes-and-clients/light-clients/>

15 49 61 63 92 <https://docs.anza.xyz/validator/geyser>
<https://docs.anza.xyz/validator/geyser>

16 27 48 76 87 https://subquery.network/doc/indexer/academy/tutorials_examples/dictionary.html
https://subquery.network/doc/indexer/academy/tutorials_examples/dictionary.html

17 <https://docs.near.org/data-infrastructure/lake-framework/near-lake>
<https://docs.near.org/data-infrastructure/lake-framework/near-lake>

18 20 37 43 69 78 85 86 <https://firehose.streamingfast.io/firehose/architecture>
<https://firehose.streamingfast.io/firehose/architecture>

26 <https://solana.com/developers/guides/advanced/confirmation>
<https://solana.com/developers/guides/advanced/confirmation>

29 67 77 <https://thegraph.com/docs/en/indexing/tap/>
<https://thegraph.com/docs/en/indexing/tap/>

31 71 <https://github.com/near/near-indexer-for-explorer>
<https://github.com/near/near-indexer-for-explorer>

32 70 84 <https://solana.com/docs/rpc>
<https://solana.com/docs/rpc>

33 <https://subquery.network/doc/indexer/build/manifest/chain-specific/polkadot.html>
<https://subquery.network/doc/indexer/build/manifest/chain-specific/polkadot.html>

34 <https://www.elastic.co/docs/manage-data/data-store/index-basics>
<https://www.elastic.co/docs/manage-data/data-store/index-basics>

35 <https://thegraph.com/docs/en/>
<https://thegraph.com/docs/en/>

36 44 51 91 <https://thegraph.com/docs/en/indexing/overview/>
<https://thegraph.com/docs/en/indexing/overview/>

38 89 https://subquery.network/doc/indexer/run_publish/optimisation.html
https://subquery.network/doc/indexer/run_publish/optimisation.html

39 50 65 <https://docs.blockscout.com/setup/information-and-settings/indexer-architecture-overview>
<https://docs.blockscout.com/setup/information-and-settings/indexer-architecture-overview>

- 40 <https://kafka.apache.org/082/getting-started/introduction/>
<https://kafka.apache.org/082/getting-started/introduction/>
- 42 <https://rocksdb.org/>
<https://rocksdb.org/>
- 47 54 56 57 <https://docs.near.org/data-infrastructure/near-indexer>
<https://docs.near.org/data-infrastructure/near-indexer>
- 55 <https://github.com/near/read-rpc>
<https://github.com/near/read-rpc>
- 59 <https://github.com/subquery/subql/blob/main/README.md>
<https://github.com/subquery/subql/blob/main/README.md>
- 62 <https://github.com/solana-labs/solana-accountsdb-plugin-postgres>
<https://github.com/solana-labs/solana-accountsdb-plugin-postgres>
- 64 73 90 <https://geth.ethereum.org/docs/fundamentals/archive>
<https://geth.ethereum.org/docs/fundamentals/archive>
- 66 <https://docs.blockscout.com/setup/microservices>
<https://docs.blockscout.com/setup/microservices>
- 72 <https://aws.amazon.com/ebs/general-purpose/>
<https://aws.amazon.com/ebs/general-purpose/>
- 74 <https://docs.aws.amazon.com/solutions/latest/live-streaming-on-aws-with-amazon-s3/cost-example-1.html>
<https://docs.aws.amazon.com/solutions/latest/live-streaming-on-aws-with-amazon-s3/cost-example-1.html>
- 75 <https://aws.amazon.com/ec2/instance-types/m7i/>
<https://aws.amazon.com/ec2/instance-types/m7i/>